
Group Aware MSI Protocol

— Chingyi Lin, Chunke Tan —

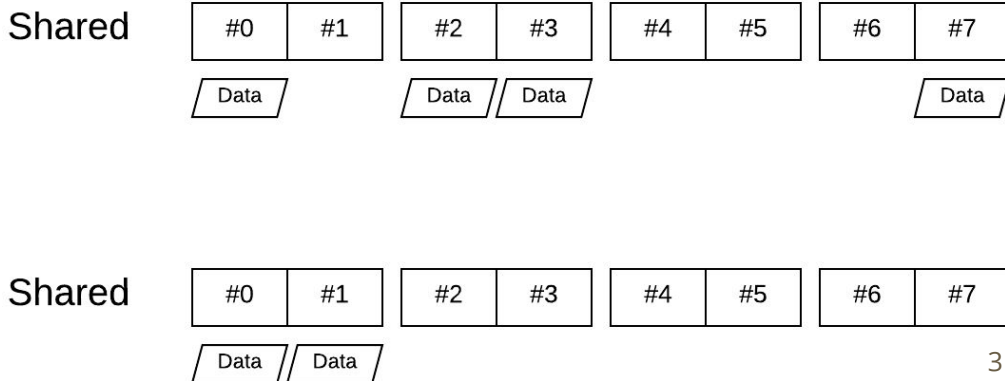
Outline

- Motivation
- Group Coherence State
- Interconnection network
- Evaluation
- Conclusion

Motivation

Cache coherence is not necessary to be held in global.

This project targets on multi-threaded program with only a subset of processor. In these cases, we hope detailed cache coherence state will improve the performance.



Group Coherence State

The flag “G” means the coherence is only held inside the group. Or this G- flag implies cacheline outside the group are all invalid

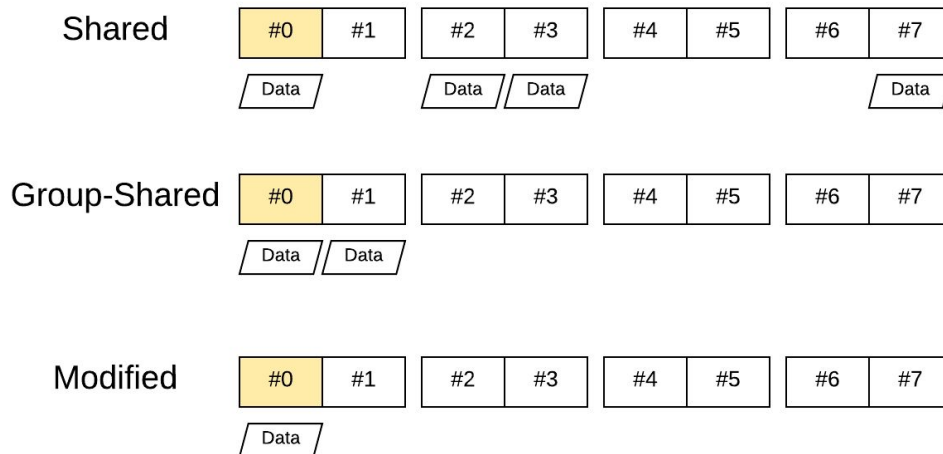
	Normal	G
Invalid	I	GI
Shared	S	GS
Modified	M	

1. Invalid state (I): Data is invalid
2. Group-invalid state (GI): Data is invalid, but it must be held by its neighbor or lower-level.
3. Shared state (S): Data is shared
4. Group-shared state (GS): Data is shared. Non-neighbors are all invalid.
5. (Group-) modified state: Modified state implies that every other is invalid.

Group Coherence State

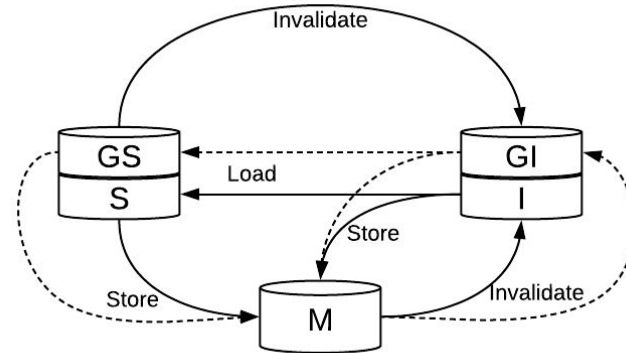
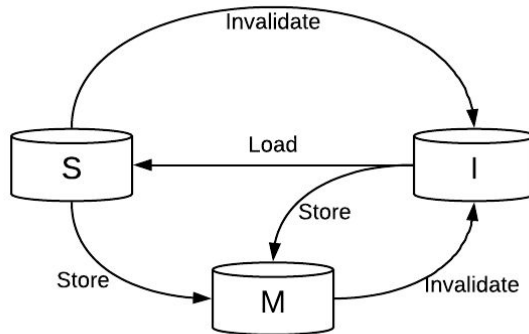
Note

1. Group-shared is a subset of shared state
2. Without G-flag, the program still works, but more inefficient



Group Coherence State

- The state transition is expanded. But the behavior of group-state is similar to the original.
- The G- flag will be given with “Reduce” request from directory, and be removed by “Expand” request.



Group Coherence State

Special case #1: All processors are in a group
(nIntras = nCore)

- Group- flag can be skipped
 - "G-" flag means "**proc outside the group**" are invalid -> Trivial
- Only three states: Modified, (G-)Shared, (G-)Invalid
- This is equivalent to MSI protocol

	Normal	G
Invalid	I	
Shared	S	
Modified	M	

Group Coherence State

Special case #2: Each processor is in a single group
(nIntras = 1)

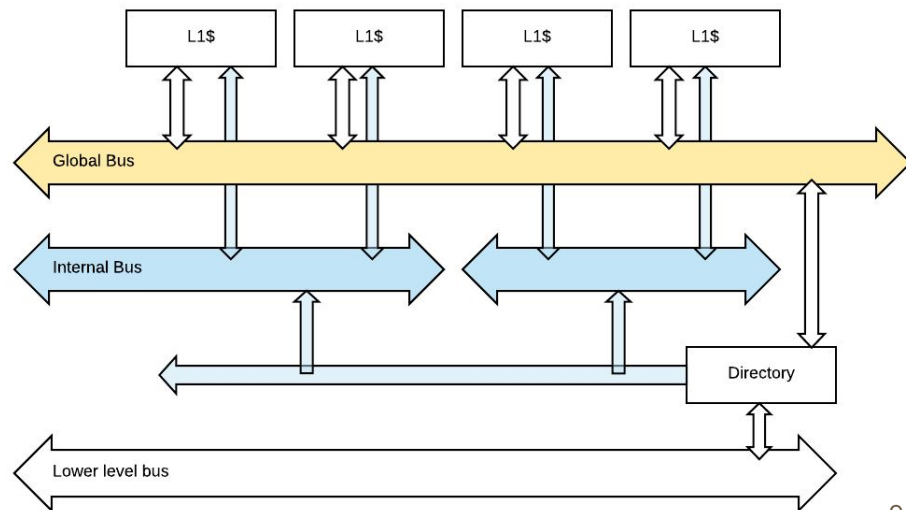
- Modified
- (Group-) Invalid: G- flag doesn't affect the behavior
- Group-shared = Exclusive state in MESI protocol
- Shared
- This is equivalent to MESI protocol

	Normal	G
Invalid	I	
Shared	S	E
Modified	M	

Interconnection network

There are two kinds of bus adhered to L1\$

- Global bus
 - Every L1\$ can transmit to another L1\$
- Internal bus
 - Can only transmit to its neighbor
- Directory
 - L1\$ -> Directory: Through global bus
 - Directory -> L1\$: Depends



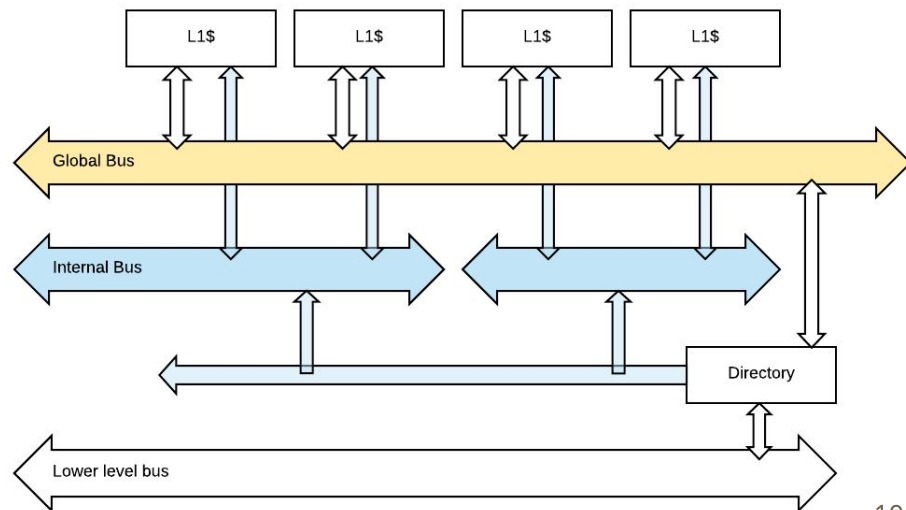
Evaluation

Parameter

- nCore: # of total core on the system
- nThread: # of thread of the program
- nIntra: # of processor in a group
 - nIntra=1,2,4 ... Fix-number group
 - nIntra=ncore, ncore/2 ... Dynamic group

Bus configuration

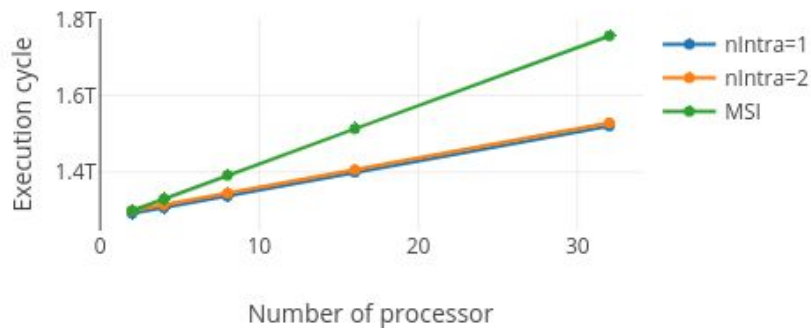
- Transmission time = $2 * n_{Intra}$



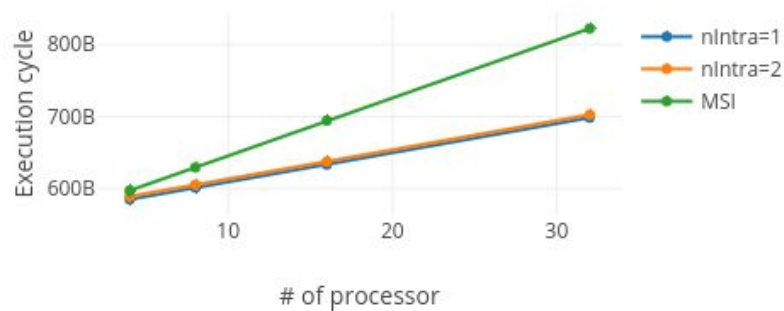
Evaluation

Execution time v.s. # of processor

2-threaded blackscholes -- Execution cycle v.s. # of processor



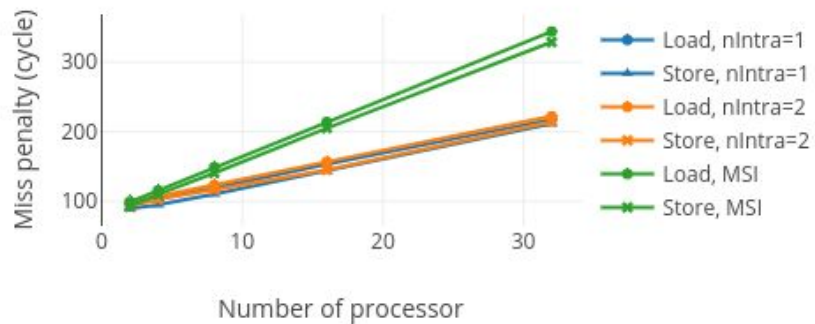
4-threaded blackscholes -- Execution cycle v.s. # of processor



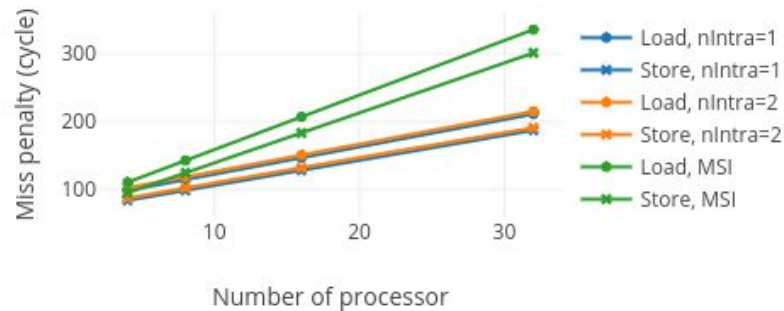
Evaluation

Miss penalty

2-threaded blackscholes -- Miss penalty



4-threaded blackscholes -- Miss penalty

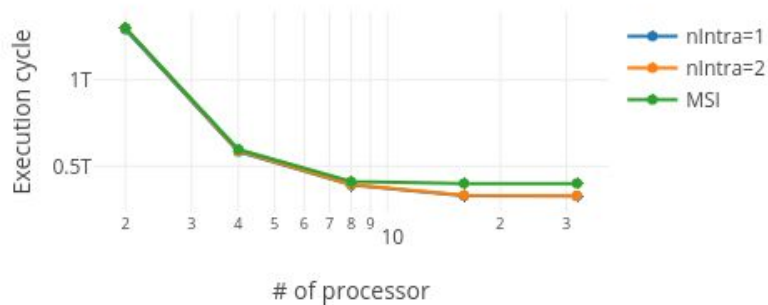


Evaluation

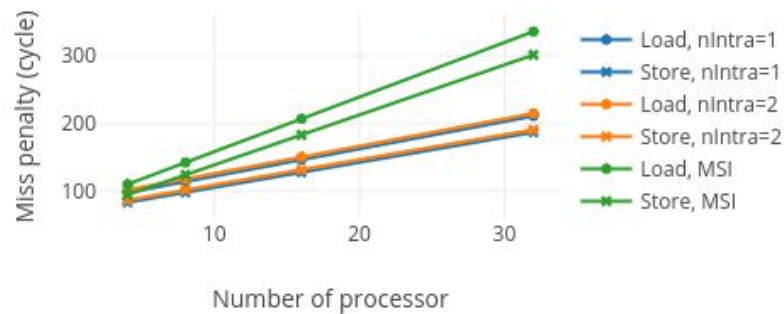
Full-thread program

- $nThread = nCore$

Full-threaded blackscholes -- Execution cycle v.s. # of processor

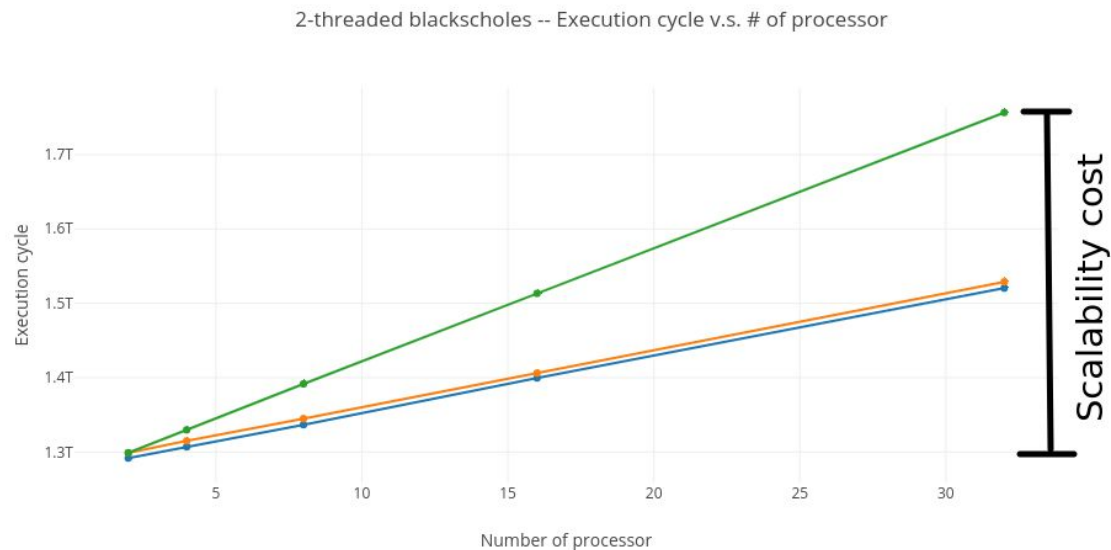


4-threaded blackscholes -- Miss penalty



Conclusion

1. The additional internal bus can decrease the miss penalty
2. Fix-numbered group saves half of scalability cost
3. G-flag works !!!

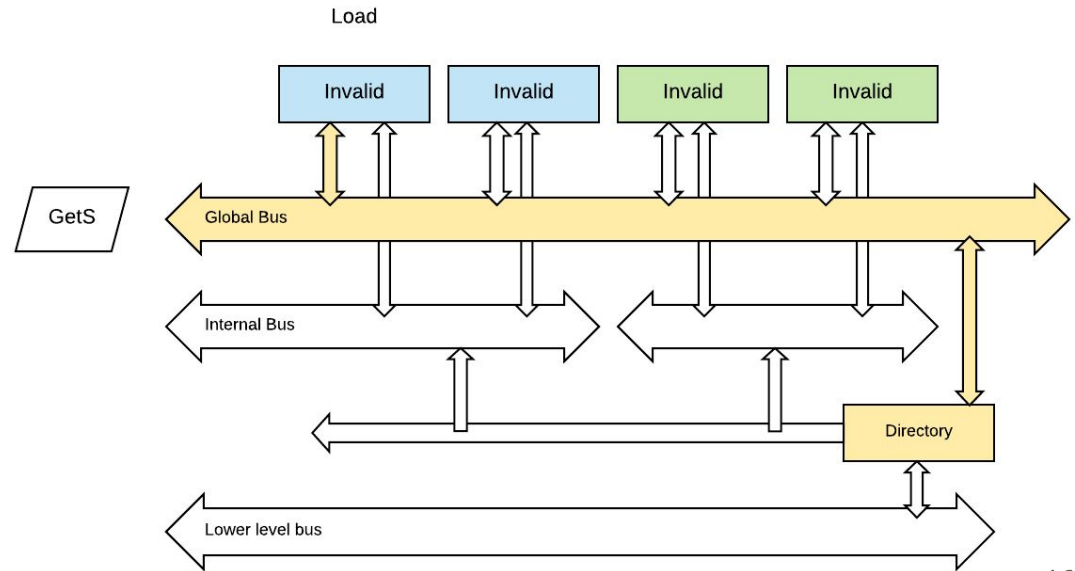


Questions?

Example -- Ask an M permission

Step (1/6): Proc #0 issue a load

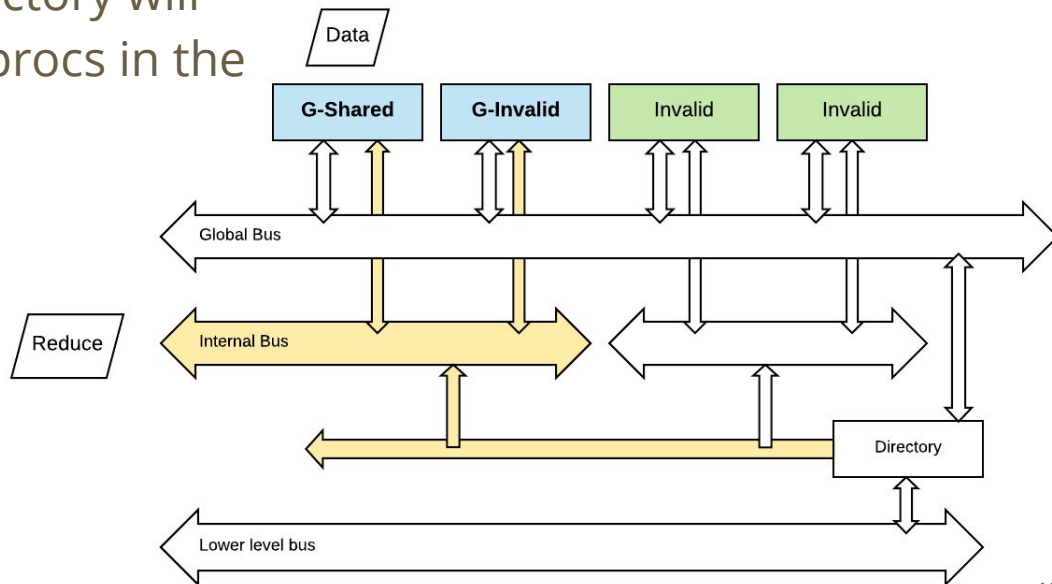
- GetS through global bus
 - From L1\$ to directory



Example -- Ask an M permission

Step (2/6): Group with proc #0 is reduced

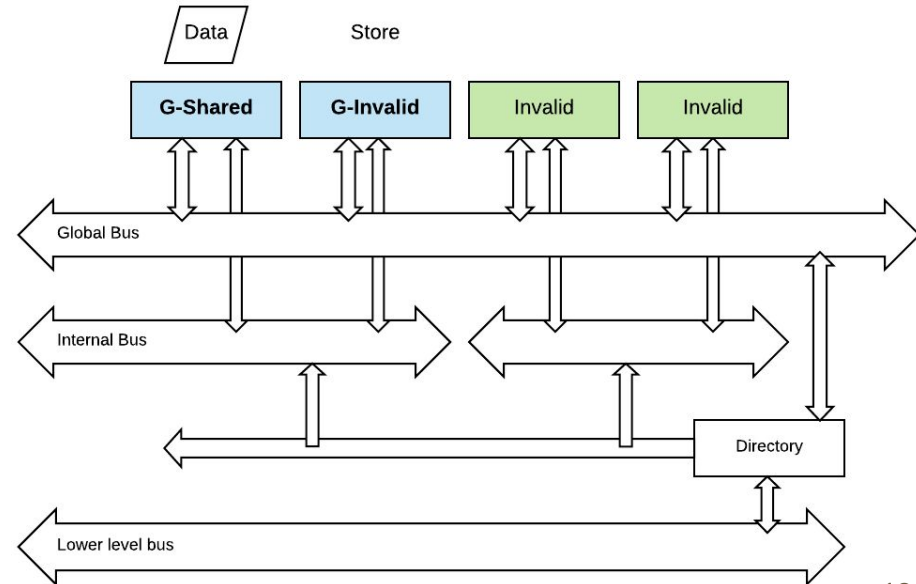
- No one has the data. So directory will send a “Reduce” request to procs in the group with proc #0
- Proc #0: Shared -> G-Shared
- Proc #1: Invalid -> G-Invalid



Example -- Ask an M permission

Step (3/6): Load from proc #0 is complete.

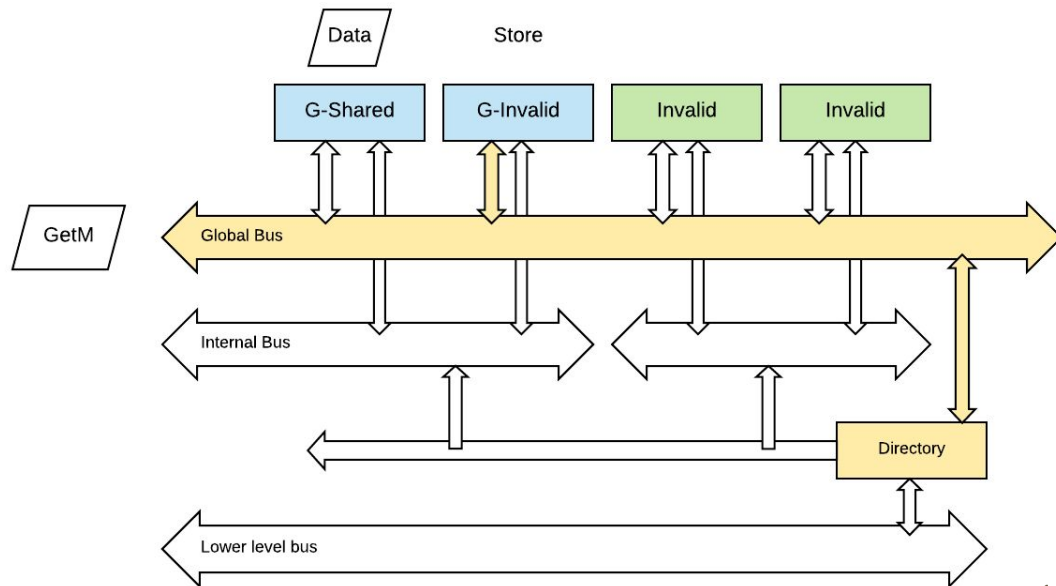
Proc #1 issues a store



Example -- Ask an M permission

Step (4/6): Proc #1 issues a store

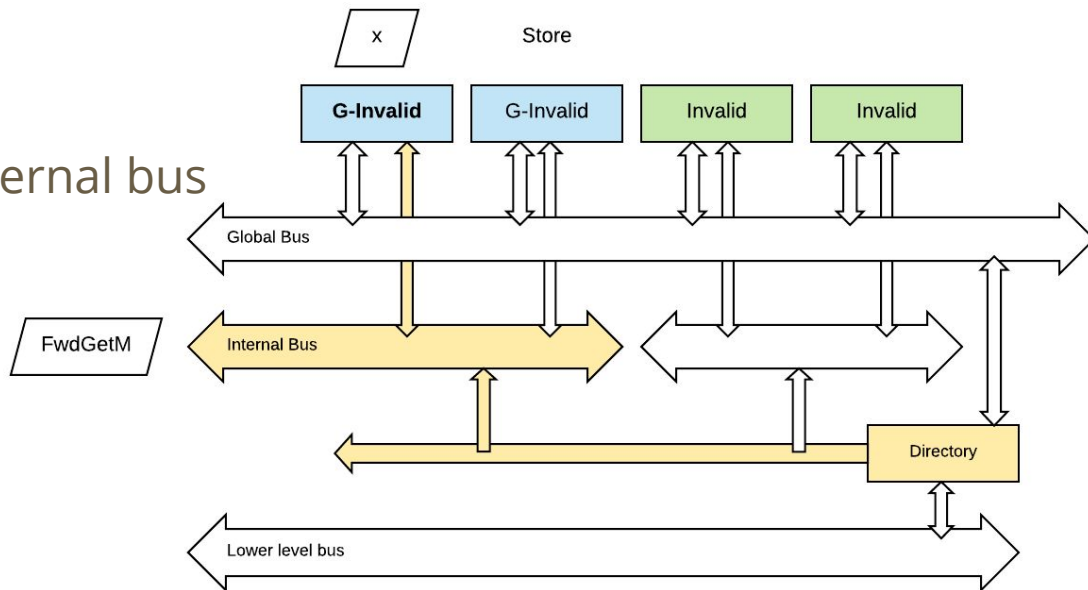
- GetM through global bus
 - From L1\$ to directory
 - With G- flag



Example -- Ask an M permission

Step (5/6): Forward this GetM to data sharers

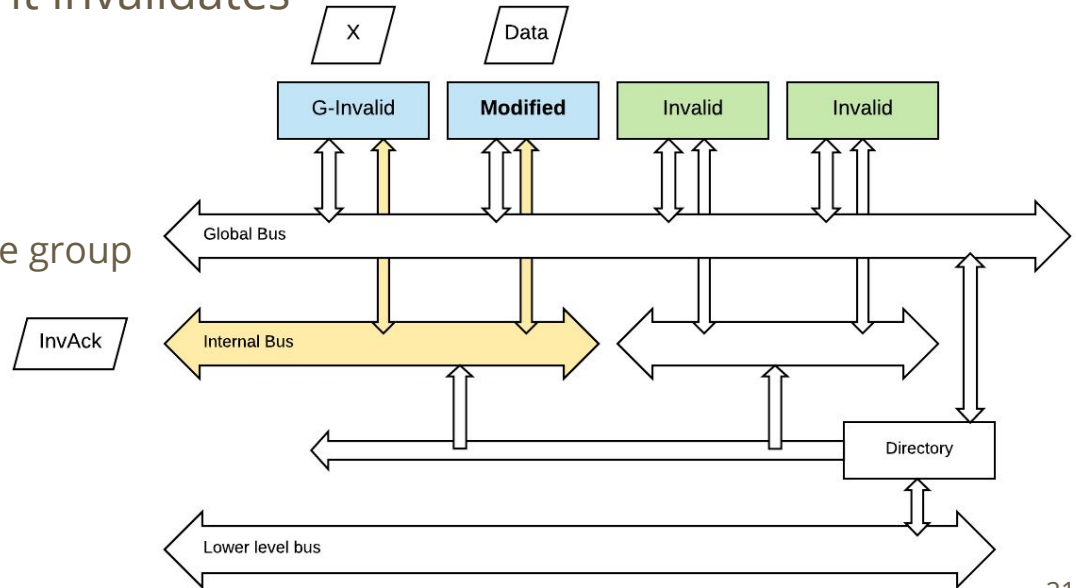
- The request is with G- flag
 - Don't care inter-proc
- Forward GetM through internal bus



Example -- Ask an M permission

Step (6/6): Send InvAck to requestor

- Proc #0 send back Ack after it invalidates the cache line.
 - The \$-state is with G-flag
 - Requestor must be in the same group
 - Proc #1 goes to Modified
-
- The diagram illustrates the communication between two processors (Proc #0 and Proc #1) and a cache. A blue box at the top represents the cache. Below it, a horizontal line represents the Global Bus. A white arrow labeled 'Global Bus' points from the cache towards the processors. Below the Global Bus, a yellow arrow labeled 'Internal Bus' points from the processors towards the cache. A white box labeled 'InvAck' is positioned between the two buses, with a line connecting it to the Global Bus arrow and another line connecting it to the Internal Bus arrow.



How nIntras affects?

